

LESSON - 20



SIKHOGE KYA???

INTRODUCTION & KEY POINTS

- INTRODUCTION
- OVERVIEW OF DATABASES
- APPLICATIONS & JOBS
- SOFTWARE INSTALLATION -
- MYSQL SERVER AND MYSQL WORK BENCH WITH DATABASE SETUP
- TABLES CREATION & FIRST QUERY

SQL STATEMENT FUNDAMENTALS

- SELECT * FROM
- SELECT DISTINCT
- COUNT, ALIAS AS
- SELECT WHERE
- LIMIT AND ORDER BY
- OPERATORS AND FILTERING
- BETWEEN, IN, LIKE AND ILIKE
- CASE IF ELSE
- COALESCE, NULLIF



JOINS & SUB QUERY

- INNER JOIN
- LEFT JOIN (LEFT OUTER JOIN)
- RIGHT JOIN (RIGHT OUTER JOIN)
- FULL JOIN (FULL OUTER JOIN)
- CROSS JOIN
- SELF JOIN
- -----
- UNION, UNION ALL, MINUS, INTERSECT
- STRING FUNCTIONS AND OPERATORS
- SUBQUERY - IMPORTANT
- DATE & TIME FUNCTIONS

GROUP BY & HAVING

- INTRODUCTION TO GROUP BY
- AGGREGATION FUNCTIONS
- GROUP BY - CHALLENGE
- HAVING
- HAVING - CHALLENGE TASKS

DATA MANIPULATION (DML)

- INSERT
- UPDATE
- DELETE
- COPY RENAME STRUCTURE
- CHECK CONSTRAINT

DATA DEFINITION LANGUAGE (DDL)

- DATA TYPES & ROWS COLUMNS
- PRIMARY FOREIGN & OTHER KEYS
- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CONSTRAINTS

ADVANCE SQL

- VIEWS / TEMP TABLES & INDEXES
- FUNCTIONS & TRIGGERS
- STORE PROCEDURES
- TRANSACTIONS
- WINDOW FUNCTIONS
- QUERY ANALYSIS TECHNIQUES

What we Learn in this Lesson ?

- What is Views ?
- Why we Use Views ?
- Create or Modify Views ?
- Drop Views ?
- How SQL / MYSQL Process Views ?
- Advance of Views -
- **Interview Questions -**

Views

A View in MySQL/SQL is a virtual table based on the result of a SELECT query. It does not store data physically, but presents data from one or more tables in a structured way.

View ?

- View is a Database Object
- View is created over an SQL Query
- View does NOT store any data
- View is like a virtual table

Use cases:

- Simplify complex queries.
- Abstract sensitive data (security).
- Provide different perspectives to different users.
- Reuse frequently needed queries.

Creating a View

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example

```
CREATE VIEW ActiveCustomers AS
SELECT customer_id, name, email
FROM customers
WHERE status = 'active';
```

Modifying a View

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example

```
CREATE OR REPLACE VIEW ActiveCustomers AS
SELECT customer_id, name
FROM customers
WHERE status = 'active' AND verified = 1;
```

Show a View

```
SELECT * FROM ActiveCustomers;
```

Drop a View

```
DROP VIEW ActiveCustomers;
```

List Views in a Database

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

Advance Options

- Insert in Views
- Update Views
- with check option

Real World Examples

Create a View: Top-Selling Products

```
CREATE VIEW TopSellingProducts AS
SELECT p.product_id, p.name, SUM(o.quantity) AS total_sold
FROM products p
JOIN orders o ON p.product_id = o.product_id
GROUP BY p.product_id, p.name
ORDER BY total_sold DESC;


SELECT * FROM TopSellingProducts WHERE total_sold > 100;
```

Interview Tips !

- Know the difference between views and tables.
- Understand when views are updatable.
- Be ready to explain performance trade-offs.
- Practice real-world examples (HR, e-commerce, school databases).